



19CSE301

Computer Networks Case Study

E-commerce Platform with Integrated TCP/UDP Communication

CB.EN.U4CSE22424	N Sai Kiran Varma
CB.EN.U4CSE22440	Soma Siva Pravallika
CB.EN.U4CSE22444	Suman Panigrahi
CB.EN.U4CSE22457	Sravani Oruganti

Project Description

The E-commerce Platform with Integrated TCP/UDP Communication and Payment Module is a robust and comprehensive application designed to facilitate the purchase and management of products through a client-server architecture. The platform aims to provide a seamless and secure experience for both consumers and administrators within an online marketplace.

The application is built using a client-server model, with the server developed using Flask and SQLAlchemy. The server manages the core functionalities of the platform, including user registration, authentication, product management, order placement, and payment processing. These operations are performed over TCP to ensure reliable and ordered communication, which is essential for maintaining the integrity of transactions and user data.

The platform supports multiple concurrent connections, allowing multiple clients to interact with the server simultaneously. The application is split into different modules, each handling specific tasks such as user management, product management, order processing, payment transactions, and real-time notifications.

In addition to the core e-commerce functionalities, the platform incorporates real-time notification features using UDP. UDP is chosen for notifications due to its lower latency and reduced overhead, making it suitable for time-sensitive updates where the occasional loss of packets is acceptable. These notifications include updates about payment confirmations, order status changes, and promotional alerts.

Overall, the E-commerce Platform with Integrated TCP/UDP Communication. The platform provides a secure and user-friendly environment for online shopping, empowering both consumers and administrators to engage in seamless e-commerce transactions.

Objectives:

1. To implement secure user authentication, data management, and payment processing using TCP.
2. To provide real-time notifications using UDP for improved user engagement.
3. To ensure seamless integration of client-side and server-side components, facilitating easy maintenance.

Technologies Used:

- Python (Flask for server-side, SQLAlchemy for database management)
- TCP/IP Protocol Suite for reliable communication
- UDP Protocol for real-time notifications
- Socket Programming for network communication

Features:

- **User Registration and Authentication:** Secure and reliable user management via TCP.
- **Product Management:** Admins can add, modify, and delete products in the database.
- **Order Placement:** Users can place orders, with transactions handled securely over TCP.
- **Payment Processing:** After order placement, users proceed to secure payment, processed via TCP.
- **Real-time Notifications:** Users receive updates about the status of their payments and orders in real-time via UDP.
- **Concurrent Connections:** The server can manage multiple client connections simultaneously, ensuring a responsive user experience.

Use Cases:

- **User Registration/Login:** A user registers or logs in to the system securely via TCP, with their details stored in the database.
- **Product Browsing:** Users can browse through the available products, with data fetched from the server via TCP.
- **Order Placement:** After browsing, users place orders which are processed through secure TCP communication.
- **Payment Processing:** After order placement, users proceed to secure payment, processed via TCP.
- **Admin Product Management:** Admins manage products through the CLI, with changes updated in the database.
- **Real-time Notifications:** Users are notified of payment confirmations, order status updates, or special offers in real-time via UDP.

Design Requirements of Transport Services

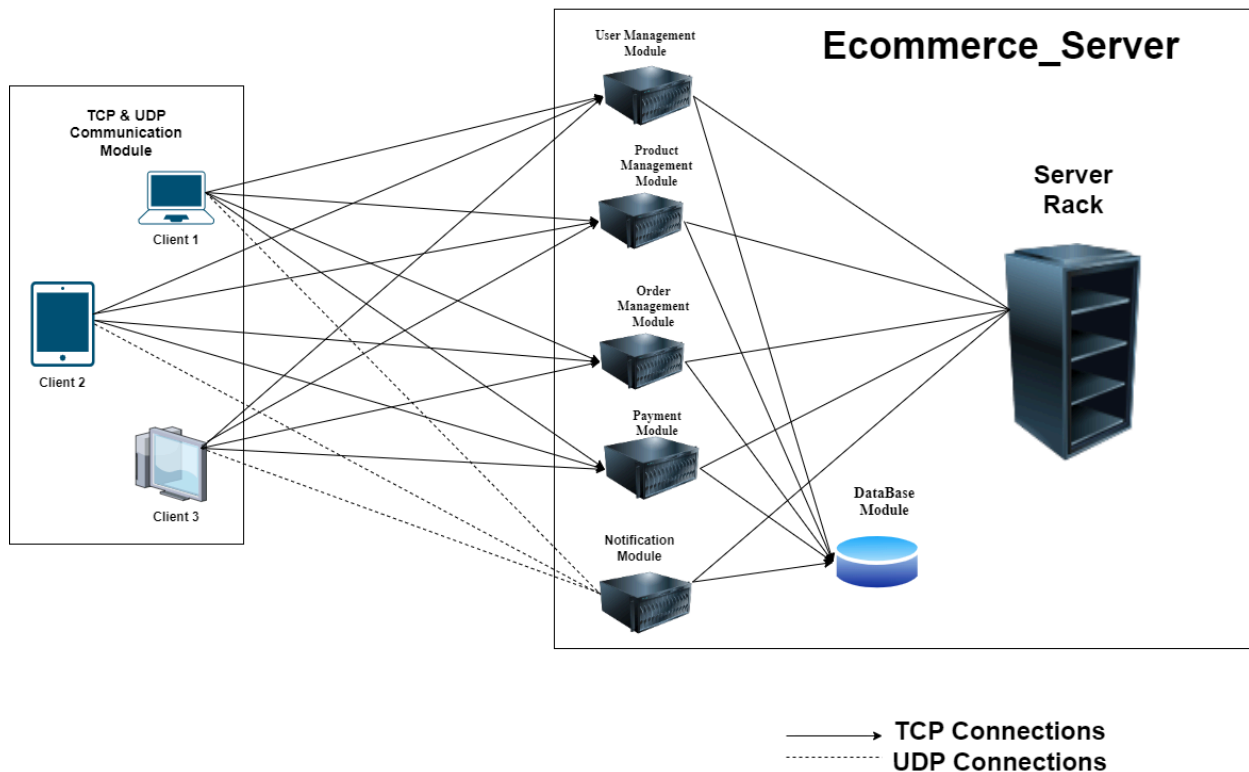
Requirement	Description
Reliable Data Transfer	Critical operations (registration, login, order placement, payment processing) require guaranteed, reliable, and ordered data transfer. TCP is used to meet this requirement.
Timing	The platform should handle user interactions (like login, order placement, payment processing) with minimal delay. UDP is used for notifications where timing is critical. TCP ensures timely data transfer for transactions.
Throughput	The platform must support multiple concurrent connections without significant performance degradation. Both TCP and UDP are optimized for high throughput in their respective use cases.
Security	Secure transmission of user data and transactions is essential. TCP is used with security protocols to ensure data integrity and protection against unauthorized access. UDP communication is limited and monitored to prevent potential security risks. Payment transactions are securely processed through encrypted TCP connections.

Architecture Diagram of the Application Modules

- **Client Side:**
 - User Interface Module (Handles user interactions, input validation)
 - TCP Communication Module (Handles requests such as registration, login, product management)
 - UDP Communication Module (Handles real-time notifications)
- **Server Side:**
 - User Management Module (Handles registration, login, and user data)
 - Product Management Module (Handles CRUD operations on products)
 - Order Management Module (Processes and manages user orders)
 - Notification Module (Handles real-time notifications using UDP)

- Database Module (Handles data storage and retrieval using SQLAlchemy)

E-Commerce Platform



Data Flow:

1. Client Sends TCP Requests:

- The client initiates actions such as registration, login, and product management by sending TCP requests to the server.

2. Server Processes Requests:

- The server receives these requests, processes them using the appropriate modules, and interacts with the database as needed (e.g., storing user data, updating product information).

3. Server Sends TCP Response:

- After processing, the server sends a response back to the client over TCP, confirming the action (e.g., successful login, product list update).

4. Real-Time Notifications via UDP:

- For events requiring immediate feedback, such as order confirmations or promotions, the server sends UDP packets to the client, which are then displayed in the CLI.

5. Handling Multiple Connections:

- The server can handle multiple TCP and UDP connections simultaneously, ensuring smooth operation for all users.

6. Error Handling:

- Any errors during TCP communication result in a clear error message to the client, while UDP notifications are sent without guaranteeing delivery but with minimal delay.

Transport Protocols and Connections Table

Module	Transport Protocol	Number of Connections
User Management	TCP	1
Product Management	TCP	1
Order Management	TCP	1
Payment Module	TCP	1
Notification Module	UDP	1

Work Plan

Part 1: Initial Setup and Core Functionality

Step 1: Setup Server and Database

- **Task 1: Initialize the Flask Application**
 - Set up a basic Flask server to handle incoming client requests.
 - Configure routing and basic server setup.
- **Task 2: Configure the Database**
 - Set up SQLAlchemy for database management.
 - Create tables for users, products, and orders.
- **Task 3: Implement Basic Server Functions**
 - Implement user registration functionality.
 - Implement user login functionality.
 - Ensure both features are fully operational and secure.

Step 2: Develop Client-Side Interface

- **Task 1: Create the Command-Line Interface (CLI)**
 - Develop a simple and intuitive CLI for user interaction.
 - Design the CLI to handle user inputs effectively.
- **Task 2: Establish TCP Communication**
 - Implement TCP communication between the client and server.
 - Focus on essential functions like registration, login, and product management.

Part 2: Advanced Features and Enhancements

Step 3: Implement Product Management

- **Task 1: Server-Side Development**
 - Create CRUD (Create, Read, Update, Delete) functionalities for product management on the server.
 - Ensure database interactions are efficient and secure.
- **Task 2: Client-Side Interaction**
 - Integrate product management features into the client interface.
 - Allow users to browse, add, update, and manage products via the CLI.

Step 4: Implement Order Management

- **Task 1: Server-Side Order Processing**
 - Develop the logic for order placement and management.
 - Ensure that orders are processed correctly and stored securely in the database.

- **Task 2: Client-Side Order Workflow**

- Add the ability for users to place and manage their orders through the CLI.
- Implement functionality for users to view order status and history.

Step 5: Implement Payment Module

- **Task 1: Implement Payment System**

- Implement a payment module that interacts with the user for payments using secure TCP communication.
- Ensure payment processing is secure and efficient.

- **Task 2: Client Payment Interface**

- Add a payment option in the CLI.
- Guide users through the payment process after placing an order.

Step 6: Implement Real-Time Notifications (UDP)

- **Task 1: Server-Side UDP Notifications**

- Develop a simple UDP server to send real-time notifications for order updates and payment confirmations.
- Ensure that the UDP server is efficient and does not interfere with the main server processes.

- **Task 2: Client-Side Notification Handling**

- Add functionality to the CLI to receive and display UDP notifications.
- Provide users with immediate feedback on their actions, such as order status updates and payment confirmations.